Componentes clave de calidad de Software

Software Quality Principles



Arellano Villaseñor Kelly Anahí 219916367

Albarran Bernal Laura Yaneri 220964065

Aguilar Nova Eric Oswaldo 219635295

Garcia Orozco Juan Pablo 220813784

Barajas Bañuelos Sandra Elizabeth 220507705

Carrera: INFO Sección: D05 Ciclo: 2026B

Materia: Calidad de Software

Maestro Rubén Adrián Gil Rivera

Tabla de contenido

01 Funcionalidad

02 Confiabilidad

03 Usabilidad

04 Rendimiento

05 Mtto. (Mantenibilidad)

06 Seguridad

07 Portabilidad

01

Funcionabilidad

¿Qué es?

La funcionalidad es la capacidad del software para proporcionar las funciones requeridas por el usuario y obtener resultados correctos, seguros y útiles.



¿Qué se evalúa?

01 **Adecuación** — el sistema incluye las funciones que el usuario necesita.

02 **Exactitud** — los resultados generados son correctos.

03 **Interoperabilidad** — puede intercambiar información con otros sistemas o formatos.

04 **Seguridad de acceso** — protege las funciones y los datos de usuarios no autorizados.

05 **Cumplimiento** — respeta los requisitos, reglas y normas establecidas.

202X

Ejemplo

NutriMaker	NutriMaker cumple su funcionalidad cuando registra correctamente los datos de un paciente y genera una dieta acorde con sus calorías, objetivo, actividad física y restricciones alimenticias. También debe permitir modificarla, eliminarla y exportarla a PDF sin perder información.
Mis Trapitos	Mis Trapitos cumple su funcionalidad cuando registra productos, ventas y clientes; descuenta automáticamente las existencias al vender; aplica promociones correctamente y muestra reportes reales de inventario y ventas.



02

Confiabilidad



Confiabilidad

Funcionar correctamente

Manera consistente



Condiciones de uso

¿Por qué es importante?

Pérdida de información



Insatisfacción de los usuarios.



Problemas de seguridad





Interrupción de servicios



Pérdidas económicas

Errores en procesos



Características de la confiabilidad

01

Ausencia de fallos frecuentes



02

Tolerancia a fallos



03

Recuperación



04

Disponibilidad



05

Consistencia





¿Cómo se puede mejorar?

- Pruebas de software
- Pruebas de rendimiento
- Corregir errores
- Monitorear después de implementación
- Copias de seguridad
- Actualizar
- Recuperación ante fallos
- Posibles errores



03

Usabilidad

202X

Definiciones

ISO 9241	"El grado en el que un producto puede ser utilizado por usuarios específicos para conseguir objetivos específicos con efectividad, eficiencia y satisfacción en un determinado contexto de uso."
ISO/IEC 9126-1	Como un proceso (enfocado en la eficacia, eficiencia y satisfacción) y como un producto (la capacidad del software de ser comprendido, aprendido, usado y de resultar atractivo).



¿Qué es?

La usabilidad es un atributo de calidad que proviene del término "user friendly", y es qué tan fácil, eficiente y agradable es para una persona utilizar un producto o sistema para lograr un objetivo.



Atributos

Facilidad de Comprensión

Fácil de entender.

Operatividad

Sencillo de manejar.

Facilidad de Aprendizaje

Rápido de asimilar.

Satisfacción

Experiencia final positiva para el usuario.

Atributos



Presentación visual apropiada

Alta productividad en poco tiempo.

Eficiencia

Prevención de equivocaciones.



Manejo de Errores

Diseño visual agradable y claro.

¿Cómo se evalúa la usabilidad?

Evaluación

Métodos empíricos

Pruebas con usuarios reales para medir su eficiencia y detectar los errores que cometen.



Métodos heurísticos

Revisión hecha por expertos basándose en reglas de diseño establecidas.



Pruebas de funcionalidad

Verificación de que cada componente cumple correctamente con su propósito.

04

Rendimiento

Rendimiento



Tiempo de respuesta



Uso de recursos



Capacidad de trabajo



Escalabilidad



Optimización



Pruebas de rendimiento

Tiempo de respuesta

01

Rapidez



02

Acciones



03

Usuario



04

Resultado

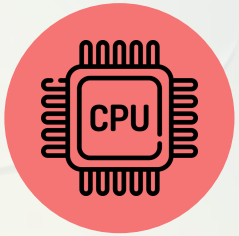


Uso de recursos

Memoria RAM

Almacenamiento

CPU / Procesador







Red



Consumo eficiente



Capacidad de trabajo

01 Usuarios

02 Operaciones

03 Procesos

04 Carga



05 Trabajo simultáneo



Escalabilidad

02

Incrementar



01

Rendimiento adecuado

03

Crecer

04

Sin perder rendimiento

Optimización

Mejoras en el código
Instrucciones más limpias, directas y estructuradas

Eliminación de operaciones innecesarias
Reducir el trabajo redundante

Optimización de consultas a bases de datos
Sin cuellos de botella

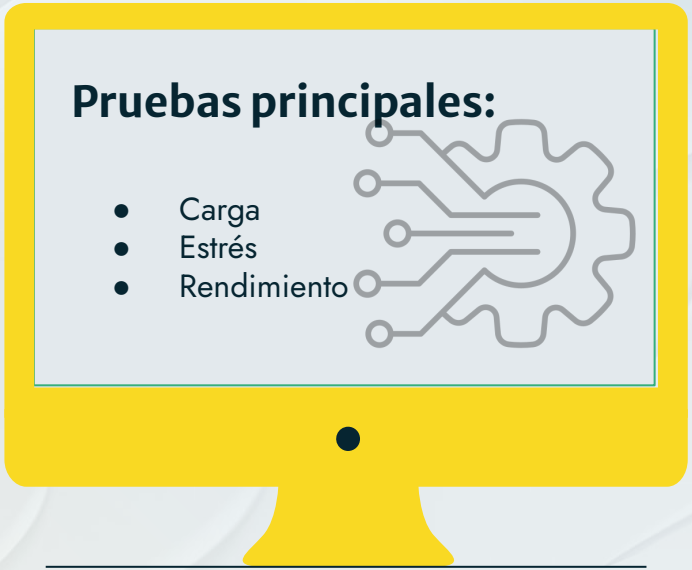
Mejor administración de memoria
Controlar el consumo de RAM

Gestión optimizada del hardware, la red y la infraestructura
Uso eficiente de recursos

Pruebas de rendimiento







05

Mantenibilidad

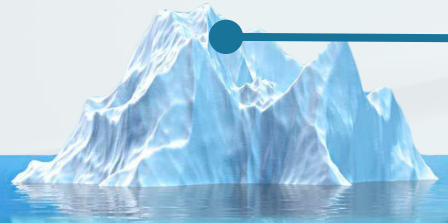
¿Qué es?

Capacidad de Evolución: El software debe escribirse pensado para cambiar y adaptarse al paso del tiempo

Flexibilidad del Código: Qué tan fácil y rápido es corregir errores, añadir funciones o cambiar de entorno sin romper el sistema



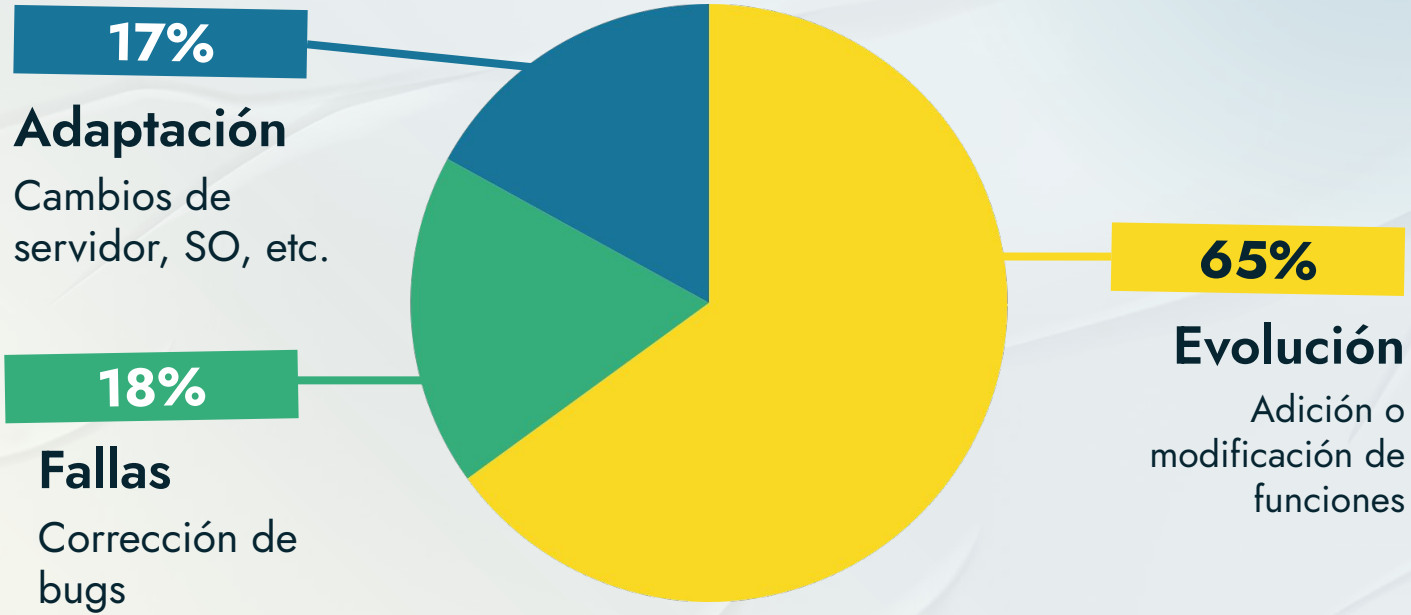
Iceberg del Software



10% - 40%
Desarrollo inicial

90% - 60%
El coste total de un software ocurre *después* de entregarlo **(en la fase de mantenimiento)**

¿En qué se divide el esfuerzo de mantenimiento?



Tipos de Mantenimiento de Software



Correctivo

Reparar fallas o bugs que reportan los usuarios



Adaptativo

Ajustar el software cuando cambia su entorno (servidores, sistemas operativos)



Perfectivo

Añadir nuevas funciones o mejoras solicitadas por el cliente



Preventivo

Limpiar y optimizar el código para que no se degrade con el tiempo

¿Cómo se mide la Mantenibilidad?



MTTC

(Mean-Time-To-Change)

El tiempo promedio que nos toma analizar, programar, probar y liberar un cambio

Spoilage

(Desperdicio)

Lo que cuesta corregir errores en producción comparado con el costo total del proyecto

Complejidad Ciclomática

Mide qué tan enredado o "espagueti" está el código

El Secreto: ¿Cómo lograrlo?

01

Componentes independientes

Diseñar módulos pequeños y autocontenidos que se puedan cambiar sin afectar a los demás

02

Evitar datos compartidos

Separar claramente a los que producen la información de los que la consumen

03

Refactorización

Mantenimiento preventivo. Limpiar, ordenar y mejorar el código constantemente sin alterar su funcionamiento

06

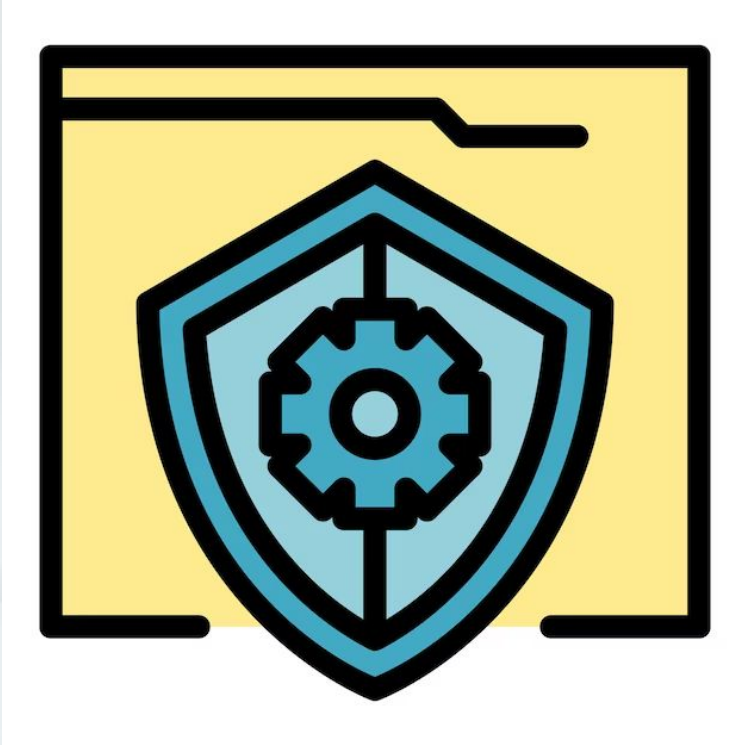
Seguridad

¿Qué es?

La **seguridad en la calidad del software** es lograr que un sistema no solo funcione bien, sino que sea capaz de **resistir ataques y proteger los datos**.

En 3 puntos clave:

- **Protección:** Mantiene la información privada y lejos de extraños.
- **Resistencia:** Evita que hackers o fallos bloqueen o dañen el sistema.
- **Confianza:** Le garantiza al usuario que sus datos están seguros en todo momento.



Los 7 pilares de la seguridad en software

01

Confidencialidad
(Que nadie no autorizado lo vea)

Ejemplo: En una app bancaria, solo tú ves el saldo de tu cuenta; ni otros clientes ni personal no autorizado pueden leer esa información.

02

Integridad
(Que nadie lo modifique sin permiso)

Ejemplo: Cuando envías una transferencia de $100, la red asegura que el monto no cambie a $1,000 en el camino hacia el receptor.

03

Disponibilidad
(Que el sistema no se caiga cuando se necesita)

Ejemplo: Que la plataforma de compras siga funcionando durante el *Buen Fin* o *Black Friday*, resistiendo ataques de tráfico masivo (DDoS).

04

Autenticación
(Demostrar quién eres)

Ejemplo: Iniciar sesión con tu contraseña y recibir un código por SMS o app autenticadora (MFA).

Los 7 pilares de la seguridad del software

05

Autorización (Lo que tienes permitido hacer)

Ejemplo: Un empleado en un hospital puede ver el expediente de un paciente, pero solo el médico puede modificar la receta.

06

Trazabilidad y Auditoría

(Saber quién hizo qué)

Ejemplo: El registro del sistema (*log*) que guarda la hora, IP y usuario exacto que borró un archivo del servidor.

07

No Repudio (Que nadie pueda negar lo que hizo)

Ejemplo: Firmar un contrato digitalmente con firma electrónica avanzada; no se puede alegar después "yo no firmé ese documento".

07

Portabilidad



¿Qué es?

Transición: Esfuerzo requerido para transferir un programa de un entorno de hardware o software a otro.

Independencia: Capacidad del software para ejecutarse sin estar "atado" a una marca, servidor o sistema operativo específico

Transición del Producto

(Modelo de McCall)

¿Hacia dónde puede "viajar" nuestro código?



Portabilidad

Qué tan fácil es mover la aplicación a otra plataforma o sistema operativo



Reusabilidad

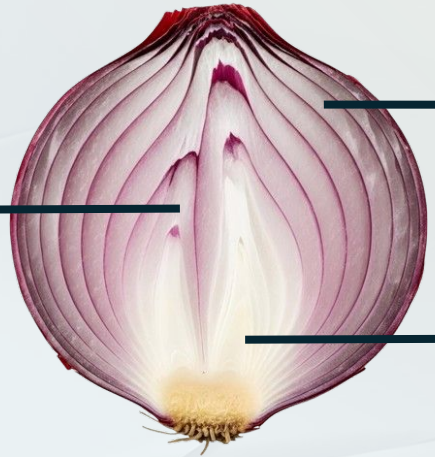
Qué tan fácil es agarrar un pedazo de ese código y usarlo en otro proyecto diferente



Interoperabilidad

Qué tan fácil es conectar este software con el de otra empresa.

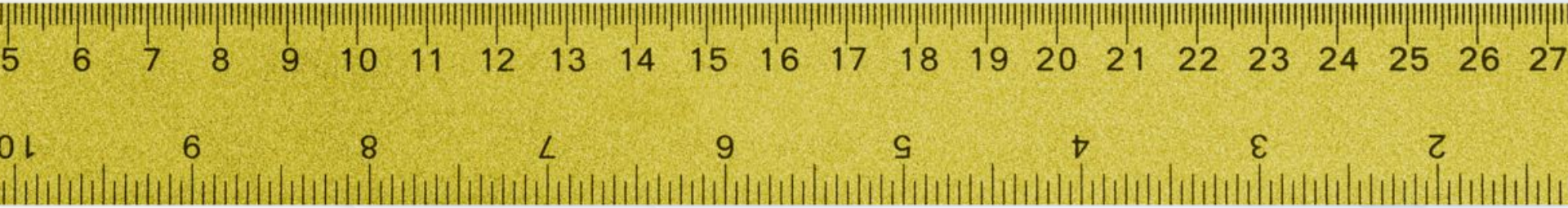
¿Cómo se logra?



Arquitectura en Capas: Aislar las dependencias de la máquina en las capas más profundas

Localización del cambio: Si cambias de entorno, solo modificas la capa inferior; la lógica del negocio se queda intacta

¿Cómo se mide la Portabilidad?



Enunciados dependientes del objetivo

Porcentaje de líneas de código que están amarradas a una plataforma específica

Número de sistemas objetivo

En cuántos entornos diferentes es capaz de operar el software actualmente

Independencia

Nivel de desacoplamiento del hardware y del sistema de software base

Los 4 Pilares de la Portabilidad *(ISO 9126)*



Adaptabilidad

Capacidad para ajustarse a nuevos entornos sin programar de nuevo

Instalabilidad

Facilidad y esfuerzo requerido para instalar el sistema en el nuevo entorno

Conformidad

Cumplimiento de los estándares de portabilidad del nuevo entorno

Reemplazabilidad

Facilidad para sustituir otro software con este

Gracias

Por su atención!

Referencias

Sommerville, I. (2011). Ingeniería de software (V. Campos Olguín, Trad.; 9.ª ed.). Pearson Educación

Pressman, R. S. (2001). Software engineering: A practitioner's approach (5th ed.). McGraw-Hill

Veloz Segura, E. A. (2022). Componentes de calidad software y su utilización en aplicaciones web. Ciencia Latina Revista Científica Multidisciplinar, 6(3), 3193–3204. https://doi.org/10.37811/cl_rcm.v6i3.2456

Mascheroni, M. A., Greiner, C. L., Dapozo, G. N., & Estayno, M. G. (2013). Ingeniería de Usabilidad. Una propuesta tecnológica para contribuir a la evaluación de la usabilidad del software. Revista Latinoamericana de Ingeniería de Software, 1(4), 125–134.

González-Sánchez, J. L., Montero-Simarro, F., & Gutiérrez-Vela, F. L. (2012). Evolución del concepto de usabilidad como indicador de calidad del software. El profesional de la información, 21(5), 529–536. https://doi.org/10.3145/epi.2012.sep.13